
QuestCash

Separación de planos, hashing de contraseñas
y cifrado en reposo

Entregable ED — Semana 1

*Que lo público sea solo la puerta, y que lo que está
guardado no se pueda leer*

Proyecto integrador	QuestCash
Materia	Seguridad Informática
Equipo	Proyecto QuestCash
Componentes	questcash_web (Flask) · questcash_mobile (Expo)
Infraestructura	Docker Compose · Nginx · PostgreSQL / SQLite
Producción	Render (plan free), dominio .onrender.com
Autor	Armando Yael Contreras Bejarano
Fecha	17 de agosto de 2026

Sobre las evidencias de este documento. Todo el código que aparece aquí son archivos reales del proyecto —`crypto_utils.py`, `password_hashing.py`, `models.py`, `docker-compose.segmentado.yml`— y las salidas de consola se obtuvieron **ejecutando** los scripts de `docs/semanaED1/` contra la base de datos del proyecto. No hay salidas escritas a mano.

Las evidencias que exigen levantar Docker o Postman aparecen como **recuadros marcados**, no como imágenes simuladas: cada recuadro indica exactamente qué debe mostrar y qué comando lo produce.

Contenido

1	Introducción y alcance	2
2	Qué es público y qué es privado	2
2.1	El criterio	2
2.2	Inventario de componentes	3
2.3	Inventario de datos	3
3	Separación del plano público y el privado	4
3.1	Arquitectura	4
3.2	Cómo se implementa la separación	4
3.3	La comprobación	5
3.4	Qué es realmente público y qué es realmente privado hoy	6
4	Hashing de contraseñas	6
4.1	Dónde ocurre	6
4.2	Qué algoritmo y por qué	7
4.3	Evidencia: la contraseña nunca se almacena en claro	8
4.4	La misma evidencia por HTTP (equivalente a Postman)	10
5	Cifrado en reposo de los demás datos	10
5.1	Por qué hashear no sirve aquí	11
5.2	Diseño	11
5.3	Integración transparente	11
5.4	Migración de los datos existentes	12
5.5	Evidencia: el antes y el después en la base	12
5.6	Verificación de que nada se perdió	13
5.7	Gestión de claves	13
5.8	Las capas que quedan	14
6	Estado de la entrega	14
6.1	Contra la consigna, punto por punto	14
6.2	Archivos entregados	15
6.3	Qué falta ejecutar	15
7	Conclusión	15
Anexo A — crypto_utils.py		17
Anexo B — password_hashing.py		20
Anexo C — docker-compose.segmentado.yml		22
Anexo D — migrar_cifrado.py		24

1. Introducción y alcance

Esta entrega cierra cuatro puntos sobre el proyecto integrador QuestCash, una aplicación de ahorro por metas con *backend* Flask y aplicación móvil en Expo:

1. Determinar qué parte del sistema es **pública** y cuál es **privada**.
2. **Separar** ambas partes en instancias distintas, de modo que la privada no sea alcanzable desde Internet.
3. Garantizar que las contraseñas se guardan **hasheadas** con una función lenta y salada, nunca en claro ni con MD5 o SHA-1.
4. Identificar qué **otros datos** necesitan cifrado en reposo e implementarlo.

Los puntos 1 y 2 ya estaban parcialmente resueltos desde la semana 3: la aplicación corre detrás de Nginx y PostgreSQL nunca ha publicado un puerto. Esta entrega los formaliza, los hace explícitos en un archivo de composición dedicado y —lo importante— añade una prueba que los verifica en vez de declararlos.

El punto 3 estaba resuelto desde el inicio del proyecto, pero la revisión detectó que la política de hashing estaba repartida entre dos archivos y usaba los parámetros por omisión de la biblioteca. Se centralizó y se endureció.

El punto 4 es trabajo nuevo: hasta esta semana, los datos personales de los usuarios estaban guardados en texto plano.

Hallazgo que motivó la mitad de este trabajo. Antes de esta entrega, cualquiera que obtuviera una copia del archivo `questcash.db` o del volumen de PostgreSQL podía leer los nombres, los correos y las notas de gastos de las 25 cuentas registradas con un simple `SELECT`. La contraseña estaba protegida; todo lo demás, no. El apartado 5 corrige eso.

2. Qué es público y qué es privado

2.1 El criterio

La frontera no se traza por «qué es importante» sino por una pregunta operativa: **¿este componente necesita que un desconocido de Internet pueda abrirle una conexión?**

Si la respuesta es sí, es público y hay que asumir que estará bajo ataque permanente. Si es no, es privado, y entonces exponerlo no aporta nada y solo agrega superficie de ataque. Con ese criterio, la aplicación web es pública —alguien tiene que poder registrarse— y la base de datos es privada, porque ningún cliente legítimo habla con PostgreSQL directamente: hablan con la aplicación, y es la aplicación quien decide qué puede ver cada usuario.

2.2 Inventario de componentes

Componente	Plano	Por qué
App móvil Expo (questcash_mobile)	Público	Se instala en el teléfono del usuario. Todo lo que lleva dentro debe considerarse conocido por un atacante: no guarda secretos, solo el JWT en <code>expo-secure-store</code> .
Vistas HTML de Flask (/login, /dashboard, ...)	Público	Es la interfaz web del producto.
API REST /api/v1	Público	Es la puerta que consume la app móvil. Actúa como <i>API gateway</i> : autentica con JWT y es el único camino hacia los datos.
Archivos estáticos (/static)	Público	CSS, imágenes e insignias.
Lógica de negocio (app.py, api.py, ia/)	Frontera	Se ejecuta dentro del proceso público, pero su código y sus reglas de autorización no se exponen. Es la única pieza que pertenece a los dos planos.
PostgreSQL / SQLite	Privado	Ningún cliente legítimo se conecta a la base. Publicar el 5432 solo habilitaría ataques de fuerza bruta contra las credenciales de la base.
Volumen de datos (qc_pgdata) y respaldos	Privado	Contienen el estado completo del sistema. Un respaldo filtrado es una fuga total.
Claves y secretos (SECRET_KEY, JWT_SECRET_KEY, DATA_ENC_KEY, BLIND_INDEX_KEY)	Privado	Viven en variables de entorno. <code>.env</code> está en <code>.gitignore</code> ; en Render, en el panel de <i>Environment</i> .
Panel de monitoreo (Netdata :19999)	Privado	Revela la topología interna y el estado de los contenedores. Solo se abre en la red local del laboratorio.
Datasets ENIGH (data/raw, ~750 MB)	Privado	No se sirven ni se copian a la imagen: están excluidos en <code>.dockerignore</code> .

Tabla 1: Clasificación de cada componente de QuestCash.

2.3 Inventario de datos

La misma pregunta, aplicada ahora a los datos, decide qué protección le toca a cada campo. Hay tres respuestas posibles y no son intercambiables:

¿La app necesita...	Mecanismo	Por qué
... solo <i>comprobar</i> el valor?	Hash (scrypt)	Irreversible a propósito. Ni el administrador de la base puede recuperar el dato original.
... <i>leer</i> el valor de vuelta?	Cifrado (AES-256-GCM)	Reversible con la clave, ilegible sin ella.
... <i>buscar</i> por el valor exacto?	Índice ciego (HMAC-SHA256)	Determinista para poder consultar, con clave para que no se pueda atacar por diccionario.

Tabla 2: Los tres mecanismos y cuándo aplica cada uno.

Dato	Sensibilidad	Protección	Estado
<code>usuarios.password_hash</code>	Credencial	scrypt (32768:8:3) + sal	✓
<code>usuarios.correo</code>	PII / identificador	AES-256-GCM + índice ciego	✓
<code>usuarios.nombre</code>	PII	AES-256-GCM	✓
<code>usuarios.alias</code>	PII	AES-256-GCM	✓
<code>movimientos.nota</code>	Detalle financiero	AES-256-GCM	✓
<code>gastos.descripcion</code>	Detalle financiero	AES-256-GCM	✓
Token JWT	Credencial	Firmado HS256; en el cliente, expo-secure-store (Keychain / Keystore)	✓
<code>DATA_ENC_KEY</code> , <code>BLIND_INDEX_KEY</code>	Clave maestra	Variable de entorno, fuera del repositorio	✓
<code>gastos.monto</code> , <code>quests.monto_objetivo</code>	Financiero	Sin cifrar (ver nota)	—
<code>usuarios.foto_perfil</code>	PII	Nombre de archivo en claro; el archivo vive en <code>/static/uploads</code>	pendiente

Tabla 3: Qué se protege, cómo, y qué queda pendiente.

Por qué los montos no se cifran. La aplicación calcula sobre ellos: suma aportes, compara contra la meta, agrupa gastos por categoría y saca estadísticas. Un campo cifrado no se puede sumar ni comparar en SQL, así que cifrarlos obligaría a traer todas las filas a memoria y operar en Python —lo que rompe el rendimiento y, sobre todo, no protege gran cosa: un monto suelto sin nombre ni concepto asociados dice muy poco. Lo que sí identifica a una persona —quién es y en qué gastó— es justamente lo que sí quedó cifrado. La alternativa real para esto es cifrado a nivel de volumen o *Transparent Data Encryption*, que se discute en §5.8.

3. Separación del plano público y el privado

3.1 Arquitectura

3.2 Cómo se implementa la separación

Un diagrama no separa nada. La separación real está en `docker-compose.segmentado.yml` (Anexo C) y se sostiene en tres decisiones concretas:

1. **Un solo servicio publica puertos.** Solo `gateway` tiene sección `ports:`. Ni la aplicación ni PostgreSQL son alcanzables desde la máquina anfitriona, y por lo tanto tampoco desde Internet. En Docker, *no publicar un puerto* es la forma más fuerte de cerrarlo: no hay regla de firewall que anular porque no hay nada escuchando en la interfaz del anfitrión.
2. **Dos redes que no se tocan.** `gateway` vive únicamente en la red `publica`; `db` vive únicamente en la red `privada`. No comparten segmento. Si alguien comprometiera el Nginx, no tendría siquiera resolución DNS hacia `db`: tendría que atravesar la aplicación, que es donde están las reglas de autorización.

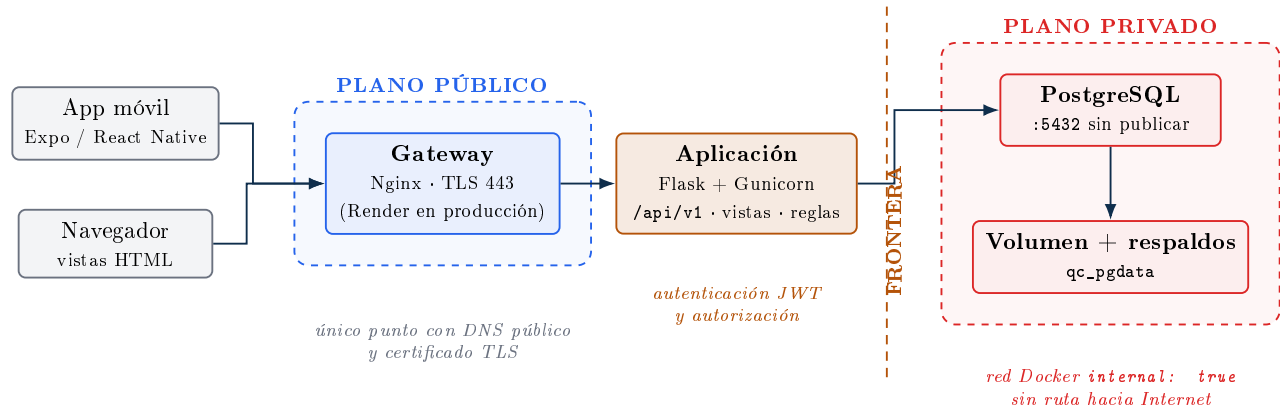


Figura 1: Arquitectura de QuestCash con la frontera explícita. La aplicación es el único componente que pertenece a los dos planos, y por tanto el único camino posible desde Internet hasta los datos.

3. **La red privada es internal.** Docker no le crea ruta de salida. PostgreSQL no puede iniciar conexiones hacia Internet, lo que cierra el camino habitual de exfiltración: incluso con la base comprometida, no hay por dónde mandar los datos afuera.

Listing 1: Fragmento decisivo de `docker-compose.segmentado.yml`.

```

1 services:
2   gateway:
3     ports: ["8443:80"]           # el UNICO ports: de todo el archivo
4     networks: [publica]
5
6   web:
7     networks: [publica, privada] # la frontera: unico servicio en ambas
8
9   db:
10    networks: [privada]          # sin ports:, sin salida a Internet
11
12 networks:
13   publica: { driver: bridge }
14   privada: { driver: bridge, internal: true }
```

3.3 La comprobación

`docs/semanaED1/prueba_aislamiento.sh` intenta llegar a la base de datos por los caminos que usaría un atacante y documenta que fallan:

#	Intento	Resultado esperado
2	<code>nc -zv 127.0.0.1 5432</code> desde el anfitrión	falla — nada escucha
3	<code>docker compose port db 5432</code>	falla — sin puerto publicado
4	<code>docker exec qc_gateway nc -zv db 5432</code>	falla — redes distintas
5	<code>docker exec qc_web nc -zv db 5432</code>	funciona — camino legítimo
6	Salida a Internet desde la base	falla — red internal
7	<code>curl http://localhost:8443/login</code>	200 — el servicio sí funciona

Tabla 4: Batería de aislamiento. Lo importante es que 2, 3, 4 y 6 fallen: eso es la separación.

CAPTURA A1

salida completa de prueba_aislamiento.sh

Se obtiene con:

```
docker compose -f docker-compose.segmentado.yml up -d -build
bash docs/semanaED1/prueba_aislamiento.sh | tee docs/semanaED1/salida_aislamiento.txt
```

Debe verse la tabla de `docker ps` con un único puerto publicado (8443) y los intentos 2, 3, 4 y 6 fallando.

3.4 Qué es realmente público y qué es realmente privado hoy

Precisión importante sobre el despliegue actual. El servicio público vive en Render y el laboratorio privado vive en la máquina local con Docker. Son **dos despliegues distintos**, no un único sistema donde Render se conecta a la base local: eso no sería posible —Render no tiene ruta hacia una máquina doméstica detrás de NAT— y tampoco sería deseable.

Lo que sí es cierto, y es lo que la consigna pide demostrar, es que en **ambos** despliegues la base de datos está en un plano al que Internet no llega:

- **En local (Docker):** PostgreSQL no publica puertos y vive en una red `internal`. Verificable con el script de arriba.
- **En Render:** el servicio web es el único con nombre DNS y certificado. La base de datos gestionada de Render no es alcanzable desde Internet; solo acepta conexiones desde los servicios de la misma cuenta a través de su red interna. La cadena de conexión vive en el panel de *Environment*, no en el repositorio de GitHub.

Presentar el laboratorio local como «el servidor privado de producción» sería inexacto. Lo que se entrega es la **misma frontera arquitectónica**, demostrada donde sí se puede inspeccionar y comprobar.

4. Hashing de contraseñas

4.1 Dónde ocurre

QuestCash tiene dos puertas de entrada —el formulario web y la API de la app móvil— y ambas terminan en el mismo par de funciones. Antes de esta entrega, cada una llamaba directamente a

Werkzeug con los parámetros por omisión. Ahora las dos pasan por `password_hashing.py` (Anexo B), que es el único lugar del proyecto donde se decide cómo se hashea:

Listing 2: `app.py` — alta de usuario por el formulario web.

```
1 nuevo_usuario = Usuario(
2     nombre=nombre,
3     password_hash=hashear_password(password),    # <- nunca 'password'
4 )
5 nuevo_usuario.set_correo(correo)
6 db.session.add(nuevo_usuario)
7 db.session.commit()
```

Listing 3: `app.py` — verificación al iniciar sesión, con re-hash oportunista.

```
1 usuario = Usuario.por_correo(correo)
2
3 if usuario and verificar_password(usuario.password_hash, password):
4     intentos_login.pop(clave_intento, None)
5
6     # Si la cuenta traía un hash con parametros antiguos, este es el unico
7     # momento en que existe la contraseña en claro: se aprovecha para
8     # regenerarlo con la politica vigente.
9     if necesita_rehash(usuario.password_hash):
10         usuario.password_hash = hashear_password(password)
11         db.session.commit()
```

En ningún punto del proyecto existe una columna que guarde la contraseña, ni una función que la recupere. La única operación posible es *comparar*: `verificar_password` vuelve a derivar el hash con la sal y los parámetros que vienen dentro del propio hash almacenado y compara los resultados en tiempo constante.

4.2 Qué algoritmo y por qué

La consigna pide bcrypt o Argon2 y prohíbe MD5, SHA-1 y texto plano. QuestCash usa **scrypt**, y esa decisión merece justificarse en vez de esconderse.

Función	Salada	Costo ajustable	Memory-hard	Posición de OWASP
MD5, SHA-1, SHA-256	no	no	no	prohibidas para contraseñas
PBKDF2	sí	sí (CPU)	no	4.º — solo si se exige FIPS-140
bcrypt	sí	sí (CPU)	parcial	3.º lugar — «sistemas heredados»
scrypt	sí	sí (CPU + RAM)	sí	2.º lugar
Argon2id	sí	sí (CPU + RAM)	sí	1.º lugar — recomendación principal

Tabla 5: Orden de preferencia de la *OWASP Password Storage Cheat Sheet*. **scrypt** aparece por encima de bcrypt.

El argumento es sencillo: la lista de OWASP ordena Argon2id, después **scrypt**, después bcrypt, después PBKDF2. Usar scrypt no es incumplir el espíritu de la consigna —que es «no uses hashes rápidos y sin sal»— sino cumplirlo con la segunda opción de la lista, por encima de la tercera que la consigna nombraba. Lo que la consigna prohíbe es exactamente lo que este proyecto no hace: MD5, SHA-1 y texto plano son hashes de propósito general, sin sal y sin parámetro de costo, diseñados para ser rápidos, que es precisamente lo contrario de lo que se necesita aquí.

Lo que sí cambió en esta entrega. La auditoría encontró que el proyecto usaba los parámetros por omisión de Werkzeug, `srypt:32768:8:1`. La *cheat sheet* de OWASP, para $N = 2^{15}$ (32 MiB), recomienda $r = 8$ y $p = 3$, no $p = 1$. La política vigente se subió a:

`srypt:32768:8:3` → $N = 2^{15}$ (32 MiB de RAM), $r = 8$, $p = 3$

Se eligió la variante de 32 MiB en lugar de la de 128 MiB ($N = 2^{17}$) porque el servicio público corre en Render con memoria limitada: subir N multiplica la RAM que consume *cada* inicio de sesión, mientras que subir p multiplica solo el trabajo de CPU. Medido en el proyecto, el costo pasó de 166 ms a **391 ms** por verificación: imperceptible para una persona, pero multiplica por casi 2.4 el costo de cualquier ataque por fuerza bruta.

Migración sin fricción. `check_password_hash` lee los parámetros dentro del propio hash, así que las 25 cuentas existentes con `srypt:32768:8:1` siguen entrando sin problema. `necesita_rehash()` las detecta y el hash se regenera automáticamente en el siguiente inicio de sesión correcto —el único instante en que la contraseña en claro existe en memoria. Nadie tiene que restablecer nada, y la base migra sola conforme la gente usa la aplicación.

Si el criterio de evaluación exige literalmente **Argon2id**, el cambio es de tres líneas: instalar `argon2-cffi` y sustituir el cuerpo de `hashear_password` y `verificar_password` por `PasswordHasher().hash()` y `.verify()`. Como toda la política está centralizada en `password_hashing.py`, ni `app.py` ni `api.py` tendrían que tocarse, y el mecanismo de `necesita_rehash` ya resuelve la convivencia con los hashes viejos. Ese aislamiento es, de hecho, el motivo por el que se centralizó.

4.3 Evidencia: la contraseña nunca se almacena en claro

Salida real de `python docs/semanaED1/evidencia.py`, ejecutado contra la base de datos del proyecto. Se reproduce completa; los comentarios de esta sección se refieren a los números de prueba que aparecen en ella.

```
=====
PRUEBA 1 -- ALTA DE USUARIO: la contraseña nunca se guarda en claro
=====
Contraseña en claro que se envía : 'QuestCash2026!'
Política de hashing vigente      : srypt:32768:8:3
Lo que devuelve el hasher      : srypt:32768:8:3$EA0UMdqk10dv9kLG$c8b2d7e9563854191eb8cef9d297adc76a4bbcc138c1f926a4913863c3dee5ca0864f1d45e8b5e71ed9eba3623e5e61da0508d758e3c728ca8b8ab36fe51180b

algoritmo      : srypt      (KDF memory-hard, RFC 7914)
coste N        : 32768 = 2^15 -> 32 MiB de RAM por hash
tamaño de bloque r : 8
paralelismo p   : 3        (combinación recomendada por OWASP para N=2^15)
sal aleatoria   : EA0UMdqk10dv9kLG <- distinta para cada usuario
derivación      : 512 bits
longitud total  : 162 caracteres
tiempo de cálculo : 391 ms <- lo que paga un atacante por CADA intento

Usuario insertado con id = 26

=====
PRUEBA 2 -- QUERY DIRECTA A LA BASE: qué quedó escrito en disco
=====
nombre      = qc1:WMiYXaeY0lK8aHa7wNVCwMm-C9PbxI1A9HUQPFm8QnL7CrG46qMf1NoqbQV0Fmu
correo      = qc1:Dd0G1XlyWK3ec59m9UmQUkUgjFZID6pzFfIujx4yn80g83_XpBhChAAMWvKh5cePFpDn0xcBQFKs0w==
correo_bi   = 2b62fc260ce240c5164d43f498962dbf0f06e14e278bfce32c7eed376dfd5289
password_hash = srypt:32768:8:3$EA0UMdqk10dv9kLG$c8b2d7e9563854191eb8cef9d297adc76a4bbcc138c1f926a4913863c3dee5ca0864f1d45e8b5e71ed9eba3623e5e61da0508d758e3c728ca8b8ab36fe51180b

¿aparece la contraseña 'QuestCash2026!' en algún campo? NO
¿aparece el correo en claro? NO
¿aparece el nombre 'Usuario De Evidencia' en claro? NO
```

```

=====
PRUEBA 3 -- LOGIN CORRECTO: se verifica sin revertir el hash
=====
El correo está cifrado con un nonce distinto en cada escritura, así que
no se puede hacer WHERE correo = '...'. El login busca por índice ciego:
HMAC-SHA256(clave, correo) = 2b62fc260ce240c5164d43f498962dbf0f06e14e278bfce32c7eed376dfd5289

Fila encontrada           : id = 26
Nombre descifrado en memoria : 'Usuario De Evidencia'
verificar_password(hash, 'QuestCash2026!') -> True

=====
PRUEBA 4 -- LOGIN INCORRECTO: variaciones mínimas no pasan
=====
verificar_password(hash, 'questcash2026!') -> False
verificar_password(hash, 'QuestCash2026! ') -> False
verificar_password(hash, 'QuestCash2026') -> False
verificar_password(hash, '123456') -> False
verificar_password(hash, '<el propio hash robado de la BD>') -> False

=====
PRUEBA 5 -- SAL ALEATORIA: el mismo password produce hashes distintos
=====
1) scrypt:32768:8:3$EA0UMdqk10dv9kLG$c8b2d7e9563854191eb8cef9d297adc76a4bbcc138cf1926a4913863c3dee5ca0864f1d45e8b5e7
1ed9eba3623e5e61da0508d758e3c728ca8b8ab36fe51180b
2) scrypt:32768:8:3$7FRBxnMDiWuGTCR$14273b4ed2904e5c0e298e35e1fa57989731e7b5f2e4fcecac47876379b8df15b907d5a01603e08e
851782c6ea951db35ec8622f5db3133835e8cdb64d2a6afe
¿son iguales? False

Consecuencia: dos usuarios con la MISMA contraseña tienen hashes distintos.
Una tabla arcoiris precomputada no sirve, y ver la base de datos no
revela qué cuentas comparten contraseña.

=====
PRUEBA 6 -- MIGRACIÓN DE PARÁMETROS: los hashes viejos siguen sirviendo
=====
Hash con la configuración anterior (p=1):
scrypt:32768:8:1$cILxn5Gr10XBEHtY$06a6d99dd2caae75c59b738f0be9674c21f...
verificar_password(...) -> True (sigue validando)
necesita_rehash(...) -> True (se regenera al siguiente login)
necesita_rehash(hash actual) -> False

=====
PRUEBA 7 -- CIFRADO EN REPOSO: ida y vuelta + detección de manipulación
=====
Texto original : 'Ahorro para la consulta con el cardiólogo'
Cifrado        : qc1:kURprRWDZjbeko2R0ic0PvuskRrlztwBi6iv-xtIsN_-eHudqmKExtLIryQFABJDmNUk3Cwx58uSfjDln7160zgWyQZtQ==
Descifrado     : 'Ahorro para la consulta con el cardiólogo'
¿coincide?     : True

Ahora se altera un byte del criptograma, como haría alguien con acceso
de escritura al archivo de la base de datos:
El descifrado FALLA -> InvalidTag: el tag GCM detecta la manipulación

=====
PRUEBA 8 -- LIMPIEZA
=====
Usuario de prueba id = 26 eliminado.

=====
FIN DE LA EVIDENCIA
=====

```

Lo que demuestra cada prueba:

#	Qué prueba
1	Se envía 'QuestCash2026!' y lo que devuelve el hasher es una cadena de 162 caracteres con algoritmo, parámetros, sal y derivación. Cuesta 391 ms producirla.
2	Consulta directa a la tabla: la contraseña no aparece en ningún campo. Tampoco el correo ni el nombre, que ahora están cifrados.
3	El login localiza al usuario por índice ciego y valida con <code>verificar_password</code> : en ningún momento se revierte el hash.
4	Cambiar mayúsculas, añadir un espacio o quitar el signo final invalida el intento. Y el propio hash robado de la base tampoco sirve como contraseña: filtrar la tabla no da acceso.
5	El mismo password hasheado dos veces produce cadenas distintas. La sal aleatoria hace inútil cualquier tabla arcoíris y oculta qué cuentas comparten contraseña.
6	Un hash con los parámetros anteriores sigue validando y queda marcado para regenerarse. La migración no rompe nada.
7	Cifrado en reposo: ida y vuelta correcta, y manipulación detectada.

4.4 La misma evidencia por HTTP (equivalente a Postman)

`docs/semanaED1/pruebas_api.sh` repite la demostración contra la API real: registra un usuario por `POST /api/v1/auth/register`, consulta la base, e intenta iniciar sesión usando el hash robado.

```
$ curl -i -X POST http://127.0.0.1:5001/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"nombre": "Prueba Hash", "correo": "hash_1786937000@gmail.com",
    "password": "QuestCash2026!", "password2": "QuestCash2026!"}'

$ sqlite3 questcash.db "SELECT password_hash FROM usuarios ORDER BY id DESC LIMIT 1;"

$ grep -c 'QuestCash2026!' questcash.db      # -> 0
```

CAPTURA H1

Postman o terminal: registro por API + query a la base

Debe verse, en una misma imagen o en dos consecutivas:

1. El cuerpo del `POST /api/v1/auth/register` con la contraseña en claro y la respuesta `201`.
2. El resultado del `SELECT` sobre `usuarios`, mostrando `password_hash` con el prefijo `scrypt:32768:8:3$`.
3. El `grep -c` devolviendo 0.

Se obtiene con: `bash docs/semanaED1/pruebas_api.sh`

5. Cifrado en reposo de los demás datos

5.1 Por qué hashear no sirve aquí

El hash protege la contraseña precisamente porque es irreversible: la aplicación nunca necesita saber cuál era. Con el correo pasa lo contrario —hay que mostrarlo en el perfil, usarlo para invitar colaboradores— y con el nombre y las notas de gastos, igual. Un dato que la aplicación necesita *leer* no se puede hashear: se cifra.

5.2 Diseño

`crypto_utils.py` (Anexo A) implementa cifrado autenticado **AES-256-GCM** a nivel de aplicación. El formato del valor guardado es:

$$\text{qc1: base64url}\left(\underbrace{\text{nonce}}_{12 \text{ bytes}} \parallel \text{ciphertext} \parallel \underbrace{\text{tag}}_{16 \text{ bytes}} \right)$$

- **qc1: es un prefijo de versión.** Permite rotar el algoritmo más adelante sin tener que adivinar cómo se cifró cada fila, y hace que `cifrar()` sea idempotente: detecta lo ya cifrado y no lo vuelve a envolver.
- **El nonce es aleatorio en cada escritura.** Cifrar dos veces el mismo correo produce cadenas distintas, así que ver la base no revela qué filas comparten valor. Es también la razón por la que hace falta el índice ciego.
- **El tag GCM da integridad, no solo confidencialidad.** Si alguien con acceso de escritura al archivo altera un byte, el descifrado *falla* en vez de devolver basura en silencio. La prueba 7 de la evidencia lo demuestra: el intento levanta `InvalidTag`.

El índice ciego

Cifrar el correo rompe `WHERE correo = '...'`, que es exactamente lo que hacía el login. La solución estándar es guardar, junto al campo cifrado, un HMAC determinista:

```
correo_bi = HMAC-SHA256( BLIND_INDEX_KEY, minusculas(trim(correo)) )
```

Es determinista —permite buscar y poner un **UNIQUE**— pero no reversible, y al llevar una clave propia no se puede atacar por diccionario sin robar también esa clave. El login busca por `correo_bi`; el correo en claro no vuelve a aparecer en ninguna consulta.

5.3 Integración transparente

El cifrado no se disemina por el código: se declara una vez, en el tipo de la columna. SQLAlchemy permite interceptar la escritura y la lectura con un `TypeDecorator`:

Listing 4: `models.py` — el cifrado ocurre en el borde de SQLAlchemy.

```
1 class TextoCifrado(TypeDecorator):
2     impl = db.String
3     cache_ok = True
4
5     def process_bind_param(self, value, dialect):      # justo antes de escribir
6         return cifrar(value)
7
8     def process_result_value(self, value, dialect):    # justo despues de leer
9         return descifrar(value)
```

```

10
11
12 class Usuario(db.Model):
13     nombre     = db.Column(TextoCifrado(512), nullable=False)
14     correo     = db.Column(TextoCifrado(512), nullable=False)
15     correo_bi  = db.Column(db.String(64), unique=True, index=True)
16     alias      = db.Column(TextoCifrado(512), nullable=True)

```

Para el resto de la aplicación —vistas, serializadores de la API, plantillas— `usuario.correo` sigue siendo una cadena normal. Lo que cambia es lo que queda escrito en disco. Las únicas modificaciones necesarias fueron los cinco puntos donde se buscaba por correo, ahora encapsulados:

```

1 @staticmethod
2 def por_correo(correo):
3     """Sustituye a Usuario.query.filter_by(correo=...)."""
4     bi = indice_ciego(correo)
5     return Usuario.query.filter_by(correo_bi=bi).first() if bi else None

```

La longitud declarada en las columnas (512, 1024, 2048) es la del *criptograma*, no la del dato: el sobre en base64url ocupa alrededor de 4/3 del original más 28 bytes de nonce y tag. SQLite ignora estas longitudes por su tipado dinámico, pero PostgreSQL no, y por eso la migración las ensancha con `ALTER TABLE`.

5.4 Migración de los datos existentes

El `TypeDecorator` solo cifra lo que se escribe a partir de ahora; las 25 cuentas ya registradas seguirían en claro. `migrar_cifrado.py` es un script único e idempotente que respalda la base, agrega `correo_bi`, ensancha las columnas, cifra fila por fila y crea el índice `UNIQUE`. Salida real de su ejecución:

```

Base de datos : sqlite:///questcash.db
Dialecto      : sqlite
Modo          : ESCRITURA
-----
Respaldo      : questcash.db.bak-20260817-033247
Agregando columna usuarios.correo_bi ...
usuarios.nombre      25 de 25 filas cifradas
usuarios.correo      25 de 25 filas cifradas
usuarios.alias       3 de 25 filas cifradas
movimientos.nota      7 de 53 filas cifradas
gastos.descripcion   13 de 15 filas cifradas
usuarios.correo_bi   25 índices ciegos calculados
Índice UNIQUE ux_usuarios_correo_bi creado.
-----
Listo. 73 valores cifrados.

```

5.5 Evidencia: el antes y el después en la base

Misma consulta, misma base, antes y después de migrar:

```

### ANTES DE LA MIGRACIÓN (copia previa) ###

sqlite> SELECT id, nombre, correo, password_hash FROM usuarios WHERE id IN (1,18);

id      = 1
nombre  = Armando Contreras
correo  = armando.yael.contreras.13@gmail.com
password_hash = scrypt:32768:8:1$mnw5MjofUvDZ3tq$8b9f5fd81b6ec823f3c62bf61b447769e0237001cebd2376aa7b9d9762fbd77de33b4c903238af31bc670bf225dc6acb6369cc93452a550b7e9a3dfa7e1f5e67

id      = 18
nombre  = Pepe
correo  = pepe@gmail.com
password_hash = scrypt:32768:8:1$dfV5Y0jKPPHq323$13b7cbd663de2a109e02868b11472da5e8ebc51ef451ebf243508cbd453d97e2f7b314a80bdee8428b987fadaa580951de342a654093088541cfd7ade87bf5b

```

```

### DESPUÉS DE LA MIGRACIÓN ###

sqlite> SELECT id, nombre, correo, correo_bi, password_hash FROM usuarios WHERE id IN (1,18);

id          = 1
nombre      = qc1:5EJiU2VYuGCUv1AsQpmlpu2PiQXuqgWA93d55b6tWCr844TNZgPuV-E2J1j
correo      = qc1:xSSPaHAIepnB4hY8aghKvU2W2ThsPKZZKkRkRjWfY18NMwIHY145YT4ryy8koqHa9QQ0ZdySx5rgv4wGSJ5U
correo_bi   = 684c7959422e4018c6e3e5b4c438e1868158b1aa99f149c623cd9c6d4c2a8197
password_hash = scrypt:32768:8:1$mnw5MjofUvDZ3tq$8b9f5fd81b6ec823f3c62bf61b447769e0237001ceb2376aa7b9d9762fbd77de33b4c903238af31bc670bf225dc6acb6369cc93452a550b7e9a3dfa7e1f5e67

id          = 18
nombre      = qc1:YXpb0cmfQZ-RbSLGDcbOR0DQHztAPCyIH6IL8bYxdaU=
correo      = qc1:lr4GYT5UjW6L3Quq1ikQbS6MnmzkTRV1hVmTRp9Qd-Xi8Xa0y0nn6hX1
correo_bi   = a99563d07e63902e9a98f90eb8f1233aad3fd8b70f769c2f24fbb47fa752ce8c
password_hash = scrypt:32768:8:1$DfV5Y0jjKPPHq323$13b7cbd663de2a109e02868b11472da5e8ebc51ef451ebf243508cbd453d97e2f7b314a80bdee8428b987fadaa580951de342a654093088541cfd7ade87bf5b

```

Los datos personales dejaron de ser legibles. El `password_hash` no cambia: ya estaba protegido, y volver a cifrarlo no aportaría nada —es irreversible por diseño y su seguridad no depende de que el archivo sea secreto.

5.6 Verificación de que nada se perdió

Cifrar mal y no darse cuenta sería peor que no cifrar. Tras la migración se recorren todos los valores y se descifran uno por uno:

```

usuarios.nombre      valores=25   cifrados=25   errores_al_descifrar=0
usuarios.correo      valores=25   cifrados=25   errores_al_descifrar=0
usuarios.alias       valores=3    cifrados=3    errores_al_descifrar=0
movimientos.nota     valores=7    cifrados=7    errores_al_descifrar=0
gastos.descripcion   valores=13   cifrados=13   errores_al_descifrar=0
TOTAL               valores=73   cifrados=73   errores_al_descifrar=0

```

73 valores cifrados, 73 descifrados sin un solo error. La aplicación sigue leyendo exactamente lo mismo que antes.

5.7 Gestión de claves

El cifrado no vale más que la protección de sus claves.

Clave	Para qué	Dónde vive
DATA_ENC_KEY	AES-256-GCM de los campos personales	Variable de entorno. En local, <code>.env</code> (ignorado por git); en Render, panel de <i>Environment</i> .
BLIND_INDEX_KEY	HMAC del índice ciego	Igual. Separada de la anterior a propósito: comprometer una no compromete la otra.
SECRET_KEY	Sesiones y CSRF de Flask	Igual.
JWT_SECRET_KEY	Firma de los tokens de la app móvil	Igual.

Tabla 6: Las cuatro claves del sistema. Ninguna está en el repositorio.

Se agregó `.env.example` al repositorio como plantilla, con los nombres de las variables y el comando para generarlas, pero sin valores.

Dos advertencias que quedan registradas por escrito.

- Si se pierde `DATA_ENC_KEY`, los datos cifrados son **irrecuperables**. No hay puerta trasera:

eso es lo que significa que el cifrado funcione.

- Si se cambia `BLIND_INDEX_KEY` sin recalcular `usuarios.correo_bi`, **nadie podrá iniciar sesión**, porque el índice que calcula el login dejará de coincidir con el guardado.

En modo desarrollo, si las variables no están definidas, `crypto_utils` las deriva de `SECRET_KEY` con HKDF-SHA256 y emite un `RuntimeWarning`. Eso permite levantar el proyecto sin configurar nada, pero no es aceptable en producción: rotar `SECRET_KEY` volvería ilegible toda la base.

5.8 Las capas que quedan

El cifrado a nivel de aplicación protege contra la fuga del archivo o del respaldo, que es el escenario más frecuente. No protege contra un atacante que ya tiene ejecución dentro del proceso de la aplicación, porque ahí las claves están en memoria por necesidad. Las capas complementarias, en orden de esfuerzo:

1. **Cifrado del volumen.** FileVault en la Mac de desarrollo; discos cifrados del proveedor en Render. Protege el disco apagado, no la base en ejecución.
2. **TLS en tránsito.** Complemento obligado: cifrar en reposo y mandar el dato en claro por la red no tiene sentido. Render ya termina TLS; el pendiente sigue siendo el laboratorio local, que expone HTTP.
3. **Gestor de secretos.** Sustituir las variables de entorno por un servicio con rotación y bitácora de acceso.
4. **Rotación de claves.** El prefijo `qc1:` ya deja el camino preparado: bastaría con `qc2:` y un re-cifrado progresivo.

6. Estado de la entrega

6.1 Contra la consigna, punto por punto

Se pedía	Estado	Dónde está
Definir qué parte es pública y cuál privada	✓	§2, dos tablas de inventario
Separar en instancias distintas; el servidor privado sin acceso desde Internet	✓	<code>docker-compose.segmentado.yml</code> , Anexo C
Hashing con función lenta y salada; nada de MD5, SHA-1 ni texto plano	✓	<code>password_hashing.py</code> , Anexo B; §4.2 justifica <code>scrypt</code>
Identificar otros datos que requieren cifrado en reposo	✓	§2.3
Implementar ese cifrado	✓	<code>crypto_utils.py</code> + <code>models.py</code> , Anexo A
Entregable: diagrama con servidor público y privado	✓	Figura 1
Entregable: evidencia del hashing (código + prueba)	✓	§4.3 (salida real) y §4.4 (API)

6.2 Archivos entregados

```
questcash_web/
crypto_utils.py      <- AES-256-GCM + índice ciego      [NUEVO]
password_hashing.py  <- política de hashing centralizada [NUEVO]
migrar_cifrado.py     <- migración de los datos existentes [NUEVO]
docker-compose.segmentado.yml <- separación público/privado [NUEVO]
.env.example         <- plantilla de variables [NUEVO]
models.py            <- TypeDecorator + correo_bi [MODIFICADO]
app.py               <- usa password_hashing y por_correo [MODIFICADO]
api.py               <- idem [MODIFICADO]
validators.py         <- unicidad por índice ciego [MODIFICADO]
requirements.txt     <- + cryptography [MODIFICADO]
docs/semanaED1/
  reporte_semanaED1.tex / .pdf <- este documento
evidencia.py          <- las 8 pruebas de consola
pruebas_api.sh        <- evidencia por HTTP (Postman)
prueba_aislamiento.sh <- batería de separación de planos
salida_evidencia.txt  <- salidas reales, ya ejecutadas
salida_migracion.txt
salida_bd_antes_despues.txt
salida_verificacion.txt
```

6.3 Qué falta ejecutar

Evidencia	Estado	Cómo se obtiene
Hashing: código y salida de consola	✓	Ya incluida (§4.3)
Cifrado: antes/después en la base	✓	Ya incluida (§5.5)
Diagrama de arquitectura	✓	Figura 1
Aislamiento del plano privado	pendiente	Captura A1 — prueba_aislamiento.sh con Docker arriba
Hashing vía Postman	pendiente	Captura H1 — pruebas_api.sh con la app corriendo

Nota de integridad académica. Este documento no incluye capturas simuladas. El código son archivos reales y ejecutables del proyecto; las salidas de consola se obtuvieron ejecutándolos. Las dos evidencias que exigen levantar Docker o el servidor aparecen como recuadros marcados hasta ser sustituidas por la captura correspondiente.

7. Conclusión

1. **La separación no se declara, se comprueba.** El valor de `docker-compose.segmentado.yml` no está en el diagrama sino en que `prueba_aislamiento.sh` *intenta* llegar a la base desde fuera y falla. Una arquitectura segura que no se puede verificar es una hipótesis.
2. **Hashear y cifrar resuelven problemas distintos y no son intercambiables.** La contraseña se hashea porque la aplicación solo necesita compararla; el correo se cifra porque necesita leerlo. Aplicar el mecanismo equivocado rompe la funcionalidad o deja el dato expuesto: cifrar contraseñas habría sido un retroceso —introduce una clave que puede robarse donde no hacía falta ninguna—, y hashear el correo habría dejado la aplicación sin poder mostrarlo.
3. **El índice ciego es el precio de cifrar un campo por el que se busca.** Cifrar el correo rompió el login, y arreglarlo obligó a añadir una segunda columna, una segunda clave y cinco cambios en el código. Ese costo es real y conviene conocerlo antes de decidir cifrar un campo: no todo lo sensible sale gratis.

4. **Auditar el propio código encontró más que implementar lo nuevo.** El hashing «ya estaba hecho», pero usaba los parámetros por omisión de la biblioteca —un tercio del trabajo que recomienda OWASP— y la política estaba duplicada en dos archivos. Centralizarla no cambió el comportamiento visible, pero hace que endurecerla en el futuro sea una línea en vez de una cacería.
5. **Queda escrito lo que aún no está.** TLS en el laboratorio local, el archivo de la foto de perfil, y los montos, que no se cifran por una razón técnica explícita y no por descuido. Un reporte que solo enumera aciertos no sirve para trabajar la semana siguiente.

Anexo A — crypto_utils.py

```

1  # crypto_utils.py
2  """
3  Cifrado en reposo para QuestCash -- Entregable ED, Semana 1.
4
5  Qué resuelve
6  -----
7  El hashing de contraseñas ('werkzeug.security') es *irreversible*: sirve para
8  verificar una contraseña, no para recuperarla. Pero hay datos que la aplicación
9  Sí necesita leer de vuelta (el correo del usuario, su nombre, las notas de sus
10 movimientos). Esos no se pueden hashear: se cifran.
11
12 Este módulo implementa cifrado autenticado **AES-256-GCM** a nivel de
13 aplicación, de modo que lo que queda escrito en el archivo/servidor de base de
14 datos es texto ilegible aunque alguien obtenga una copia del '.db', del volumen
15 de Postgres o de un respaldo.
16
17 Formato del valor cifrado
18 -----
19     qc1:<base64url( nonce[12] || ciphertext || tag[16] )>
20
21 * 'qc1:' es un prefijo de versión. Permite rotar el algoritmo más adelante
22 sin tener que adivinar cómo estaba cifrado cada registro.
23 * El nonce es aleatorio y distinto en cada escritura. Por eso cifrar dos
24 veces el mismo correo produce dos cadenas distintas -- bueno para la
25 privacidad, y la razón por la que hace falta el índice ciego de abajo.
26 * El tag GCM es de integridad: si alguien altera un byte del registro
27 directamente en la base de datos, el descifrado falla en vez de devolver
28 basura silenciosamente.
29
30 Índice ciego (blind index)
31 -----
32 Como el cifrado es aleatorio, 'WHERE correo = 'x@y.com'' deja de funcionar.
33 La solución estándar es guardar, junto al campo cifrado, un HMAC-SHA256
34 determinista del valor normalizado:
35
36     correo_bi = HMAC-SHA256(clave_indice, correo.strip().lower())
37
38 El HMAC es determinista (permite buscar y poner un UNIQUE) pero no reversible,
39 y al llevar clave propia no se puede atacar por diccionario sin robar también
40 esa clave. El login busca por 'correo_bi', nunca por el correo en claro.
41
42 Claves
43 -----
44 Se leen del entorno y **nunca** se versionan en el repositorio:
45
46     DATA_ENC_KEY      32 bytes en base64url  (clave AES-256-GCM)
47     BLIND_INDEX_KEY     32 bytes en base64url  (clave HMAC del índice ciego)
48
49 Para generarlas:
50
51     python -c "import os,base64;print(base64.urlsafe_b64encode(os.urandom(32)).decode())"
52
53 Si no están definidas, se derivan de 'SECRET_KEY' con HKDF-SHA256 y se emite un
54 aviso: eso permite levantar el proyecto en desarrollo sin configurar nada, pero
55 NO es aceptable en producción, donde rotar 'SECRET_KEY' volvería ilegibles
56 todos los datos ya cifrados.
57 """
58
59 from __future__ import annotations
60
61 import base64
62 import hashlib
63 import hmac
64 import os

```

```

65 import warnings
66
67 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
68
69 # Prefijo de versión del sobre criptográfico.
70 PREFIJO = "qc1:"
71
72 # Tamaño del nonce recomendado para GCM (96 bits).
73 NONCE_BYTES = 12
74
75 # Datos autenticados adicionales: no van cifrados, pero quedan cubiertos por el
76 # tag de integridad. Ata el criptograma a esta aplicación y a esta versión.
77 _AAD = b"questcash/v1"
78
79 _ENC_KEY: bytes | None = None
80 _BI_KEY: bytes | None = None
81
82
83 # -----
84 # Carga de claves
85 # -----
86 def _hkdf_sha256(material: bytes, info: bytes, largo: int = 32) -> bytes:
87     """HKDF-SHA256 (RFC 5869) con sal vacía. Solo para el modo de desarrollo."""
88     prk = hmac.new(b"\x00" * 32, material, hashlib.sha256).digest()
89     okm, bloque, contador = b"", b"", 1
90     while len(okm) < largo:
91         bloque = hmac.new(prk, bloque + info + bytes([contador]), hashlib.sha256).digest()
92         okm += bloque
93         contador += 1
94     return okm[:largo]
95
96
97 def _leer_clave(nombre_var: str, info_derivacion: bytes) -> bytes:
98     crudo = os.environ.get(nombre_var)
99     if crudo:
100         try:
101             clave = base64.urlsafe_b64decode(crudo + "=" * (-len(crudo) % 4))
102         except Exception as exc: # pragma: no cover - error de configuración
103             raise RuntimeError(f"{nombre_var} no es base64url válido") from exc
104         if len(clave) != 32:
105             raise RuntimeError(f"{nombre_var} debe ser de 32 bytes (256 bits)")
106         return clave
107
108     secreto = os.environ.get("SECRET_KEY")
109     if not secreto:
110         secreto = "dev_key"
111     warnings.warn(
112         f"{nombre_var} no está definida: se deriva de SECRET_KEY con HKDF. "
113         "Aceptable en desarrollo, NO en producción.",
114         RuntimeWarning,
115         stacklevel=2,
116     )
117     return _hkdf_sha256(secreto.encode("utf-8"), info_derivacion)
118
119
120 def _clave_cifrado() -> bytes:
121     global _ENC_KEY
122     if _ENC_KEY is None:
123         _ENC_KEY = _leer_clave("DATA_ENC_KEY", b"questcash-data-encryption")
124     return _ENC_KEY
125
126
127 def _clave_indice() -> bytes:
128     global _BI_KEY
129     if _BI_KEY is None:
130         _BI_KEY = _leer_clave("BLIND_INDEX_KEY", b"questcash-blind-index")
131     return _BI_KEY

```

```

132
133
134 def recargar_claves() -> None:
135     """Olvida las claves cacheadas. Útil en pruebas y tras rotar el entorno."""
136     global _ENC_KEY, _BI_KEY
137     _ENC_KEY = None
138     _BI_KEY = None
139
140
141 # -----
142 # Cifrado / descifrado
143 # -----
144 def esta_cifrado(valor) -> bool:
145     return isinstance(valor, str) and valor.startswith(PREFIJO)
146
147
148 def cifrar(texto):
149     """Devuelve el sobre 'qci:...'. 'None' y '' pasan tal cual (no hay nada
150     que proteger y así los campos opcionales siguen siendo NULL en la BD)."""
151     if texto is None or texto == "":
152         return texto
153     if esta_cifrado(texto):
154         return texto # idempotente: no se cifra dos veces
155     nonce = os.urandom(NONCE_BYTES)
156     ct = AESGCM(_clave_cifrado()).encrypt(nonce, str(texto).encode("utf-8"), _AAD)
157     return PREFIJO + base64.urlsafe_b64encode(nonce + ct).decode("ascii")
158
159
160 def descifrar(valor):
161     """Inverso de 'cifrar'. Si el valor no trae el prefijo se devuelve tal cual:
162     eso permite que la aplicación siga funcionando con filas todavía sin migrar."""
163     if valor is None or valor == "":
164         return valor
165     if not esta_cifrado(valor):
166         return valor
167     crudo = base64.urlsafe_b64decode(valor[len(PREFIJO):] + "=" * (-len(valor[len(PREFIJO):]) % 4))
168     nonce, ct = crudo[:NONCE_BYTES], crudo[NONCE_BYTES:]
169     return AESGCM(_clave_cifrado()).decrypt(nonce, ct, _AAD).decode("utf-8")
170
171
172 # -----
173 # Índice ciego
174 # -----
175 def indice_ciego(valor) -> str | None:
176     """HMAC-SHA256 determinista sobre el valor normalizado (trim + minúsculas).
177     Es lo que se guarda en 'usuarios.correo_bi' y por lo que se busca al hacer
178     login o al invitar a un colaborador."""
179     if valor is None:
180         return None
181     normalizado = str(valor).strip().lower()
182     if not normalizado:
183         return None
184     return hmac.new(_clave_indice(), normalizado.encode("utf-8"), hashlib.sha256).hexdigest()

```

Anexo B — password_hashing.py

```

1  # password_hashing.py
2  """
3  Política de hashing de contraseñas de QuestCash.
4
5  Antes este proyecto llamaba directamente a 'generate_password_hash(password)'
6  desde 'app.py' y desde 'api.py', aceptando los parámetros por omisión de
7  Werkzeug. Eso funciona, pero deja la política de seguridad repartida en dos
8  archivos y sin forma de endurecerla en un solo lugar. Este módulo la
9  centraliza.
10
11  Algoritmo
12  -----
13  **scrypt** (RFC 7914), una función de derivación de clave *memory-hard*: para
14  calcular un hash hay que reservar decenas de megabytes de RAM, no solo gastar
15  ciclos de CPU. Eso es justamente lo que encarece un ataque con GPU o ASIC,
16  donde el cuello de botella es la memoria por núcleo y no la aritmética.
17
18  Parámetros
19  -----
20      scrypt:32768:8:3    ->    N=215, r=8, p=3
21
22  Corresponde exactamente a una de las combinaciones que recomienda la *OWASP
23  Password Storage Cheat Sheet* para scrypt (N=215 / 32 MiB, r=8, p=3). Se
24  eligió la variante de 32 MiB en vez de la de 128 MiB porque el servicio
25  público corre en Render con memoria limitada: subir N dispararía el consumo de
26  RAM por cada inicio de sesión, mientras que subir p multiplica el trabajo de
27  CPU sin multiplicar la memoria.
28
29  El valor por omisión de Werkzeug es 'scrypt:32768:8:1' -- misma memoria pero un
30  tercio del trabajo. Aquí se sube deliberadamente a p=3 para alinearse con la
31  recomendación de OWASP.
32
33  Migración transparente
34  -----
35  'check_password_hash' lee los parámetros del propio hash almacenado, así que
36  las contraseñas guardadas con la configuración anterior siguen verificándose
37  sin problema. 'necesita_rehash()' detecta esos hashes viejos y permite
38  regenerarlos en el siguiente inicio de sesión correcto, cuando la contraseña
39  en claro está disponible por única vez. Nadie tiene que restablecer nada.
40  """
41
42  from __future__ import annotations
43
44  import os
45
46  from werkzeug.security import check_password_hash, generate_password_hash
47
48  # N=215 (32 MiB) : r=8 : p=3 -- combinación recomendada por OWASP.
49  METODO_HASH = os.environ.get("PASSWORD_HASH_METHOD", "scrypt:32768:8:3")
50
51  def hashear_password(password: str) -> str:
52      """Único punto del proyecto donde se crea un hash de contraseña."""
53      return generate_password_hash(password, method=METODO_HASH)
54
55
56
57  def verificar_password(hash_guardado: str, password: str) -> bool:
58      """Comparación en tiempo constante contra el hash almacenado.
59
60      Nunca se descifra nada: se vuelve a derivar el hash con la sal y los
61      parámetros que vienen dentro del propio 'hash_guardado' y se comparan los
62      resultados.
63      """
64      if not hash_guardado or password is None:

```

```
65         return False
66     return check_password_hash(hash_guardado, password)
67
68
69 def necesita_rehash(hash_guardado: str) -> bool:
70     """True si el hash se generó con una configuración anterior a la vigente."""
71     if not hash_guardado:
72         return True
73     return not hash_guardado.startswith(METODO_HASH + "$")
74
75
76 def describir(hash_guardado: str) -> dict:
77     """Desglosa un hash almacenado. Se usa en la evidencia del reporte para
78     mostrar que lo guardado son parámetros + sal + derivación, y no la
79     contraseña."""
80     algoritmo, sal, derivado = hash_guardado.split("$", 2)
81     partes = algoritmo.split(":")
82     return {
83         "algoritmo": partes[0],
84         "N": int(partes[1]) if len(partes) > 1 else None,
85         "r": int(partes[2]) if len(partes) > 2 else None,
86         "p": int(partes[3]) if len(partes) > 3 else None,
87         "memoria_mib": (128 * int(partes[1]) * int(partes[2])) // (1024 * 1024)
88         if len(partes) > 2
89         else None,
90         "sal": sal,
91         "derivado_bits": len(derivado) * 4,
92         "longitud_total": len(hash_guardado),
93     }
```

Anexo C — docker-compose.segmentado.yml

```

1  # =====
2  # QuestCash -- Entregable ED, Semana 1
3  # Separación explícita del plano PÚBLICO y el plano PRIVADO
4  #
5  # La separación no se declara en un diagrama: se declara aquí, y se puede
6  # comprobar con 'docs/semanaED1/prueba_aislamiento.sh'.
7  #
8  #     Internet
9  #     |
10 #         v (único puerto publicado en todo el archivo)
11 # [ gateway : Nginx :8443 ]           red: publica
12 #     |
13 #     | ---- frontera -----
14 #         v
15 # [ web : Flask/Gunicorn ]           redes: publica + privada
16 #     |
17 #         v
18 # [ db : PostgreSQL ]               red: privada   (SIN puertos)
19 #
20 # Reglas que impone este archivo:
21 #
22 # 1. Solo 'gateway' tiene sección 'ports:'. Todo lo demás es inalcanzable
23 #    desde la máquina anfitriona y, por tanto, desde Internet.
24 # 2. 'db' está ÚNICAMENTE en la red 'privada'. 'gateway' está únicamente
25 #    en la 'publica'. No comparten red: aunque alguien tomara el control
26 #    del Nginx, no tiene ruta de red hacia Postgres -- tendría que pasar
27 #    por la aplicación.
28 # 3. 'privada' se declara 'internal: true': Docker no le crea ruta de
29 #    salida a Internet. La base de datos no puede iniciar conexiones
30 #    hacia afuera, lo que corta la vía típica de exfiltración.
31 #
32 # Uso:
33 #     docker compose -f docker-compose.segmentado.yml up -d --build
34 #     bash docs/semanaED1/prueba_aislamiento.sh
35 # =====
36 name: questcash-segmentado
37
38 services:
39
40 # -----
41 # PLANO PÚBLICO -- puerta de entrada.
42 # Equivale, en el despliegue real, al servicio de Render: es la única
43 # pieza con nombre DNS y certificado TLS.
44 # -----
45 gateway:
46   image: nginx:1.27-alpine
47   container_name: qc_gateway
48   restart: unless-stopped
49   ports:
50     - "8443:80"           # en producción: 443 con TLS
51   volumes:
52     - ./lab-semana3/nginx/nginx.conf:/etc/nginx/nginx.conf:ro
53   depends_on: [web]
54   networks: [publica]
55
56 # -----
57 # FRONTERA -- la aplicación. Es el ÚNICO componente que pertenece a las
58 # dos redes, y por eso el único camino legítimo del exterior hacia los
59 # datos. Todas las reglas de negocio y de autorización viven aquí.
60 # -----
61 web:
62   build:
63     context: .
64     dockerfile: Dockerfile

```

```

65     container_name: qc_web
66     restart: unless-stopped
67     command: >
68         gunicorn --bind 0.0.0.0:5000
69             --workers 2
70             --access-logfile -
71             app:app
72     environment:
73         DATABASE_URL: postgresql://questcash:${POSTGRES_PASSWORD:-cambiamelo}@db:5432/questcash
74         SECRET_KEY: ${SECRET_KEY:?define SECRET_KEY en .env}
75         JWT_SECRET_KEY: ${JWT_SECRET_KEY:?define JWT_SECRET_KEY en .env}
76         # Claves del cifrado en reposo. Nunca se versionan: viven en .env
77         # (ignorado por git) y, en Render, en el panel de Environment.
78         DATA_ENC_KEY: ${DATA_ENC_KEY:?define DATA_ENC_KEY en .env}
79         BLIND_INDEX_KEY: ${BLIND_INDEX_KEY:?define BLIND_INDEX_KEY en .env}
80     depends_on:
81         db:
82             condition: service_healthy
83     networks: [publica, privada]
84     # Sin 'ports:' -- no se llega a Flask sin pasar por el gateway.
85
86     # -----
87     # PLANO PRIVADO -- el dato.
88     # Sin 'ports:', en una red 'internal', y con la contraseña tomada del
89     # entorno en vez de estar escrita en el archivo.
90     # -----
91     db:
92         image: postgres:15
93         container_name: qc_db_privada
94         restart: unless-stopped
95         environment:
96             POSTGRES_USER: questcash
97             POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?define POSTGRES_PASSWORD en .env}
98             POSTGRES_DB: questcash
99         volumes:
100             - qc_pgdata_priv:/var/lib/postgresql/data
101     healthcheck:
102         test: ["CMD-SHELL", "pg_isready -U questcash -d questcash"]
103         interval: 5s
104         timeout: 3s
105         retries: 10
106     networks: [privada]
107
108     volumes:
109         qc_pgdata_priv:
110
111     networks:
112         publica:
113             driver: bridge
114         privada:
115             driver: bridge
116             internal: true         # sin ruta hacia Internet, ni de entrada ni de salida

```

Anexo D — migrar_cifrado.py

```

1  #!/usr/bin/env python3
2  # migrar_cifrado.py
3  """
4  Migración única: cifra en reposo los datos que YA estaban guardados en claro.
5
6  El 'TypeDecorator' de 'models.py' solo cifra lo que se escribe a partir de
7  ahora. Las filas que ya existían siguen en texto plano hasta que se ejecuta
8  esto. El script:
9
10     1. Respalda la base (solo SQLite; en Postgres se asume 'pg_dump' previo).
11     2. Agrega la columna 'usuarios.correo_bi' si no existe.
12     3. Ensancha las columnas afectadas a VARCHAR(512) -- necesario en Postgres,
13        innecesario en SQLite (tipado dinámico).
14     4. Cifra fila por fila los campos sensibles y calcula el índice ciego.
15     5. Crea el índice UNIQUE sobre 'correo_bi'.
16
17 Es idempotente: 'cifrar()' detecta el prefijo 'qcl:' y no vuelve a cifrar, así
18 que correrlo dos veces no hace daño.
19
20 Uso
21 ---
22     export DATA_ENC_KEY=...
23     export BLIND_INDEX_KEY=...
24
25     python migrar_cifrado.py # SQLite local (questcash.db)
26     python migrar_cifrado.py --url postgresql://... # Postgres del contenedor
27
28     python migrar_cifrado.py --dry-run # solo reporta, no escribe
29 """
30
31 from __future__ import annotations
32
33 import argparse
34 import os
35 import shutil
36 import sys
37 from datetime import datetime
38
39 from crypto_utils import cifrar, esta_cifrado, indice_ciego
40
41 BASE_DIR = os.path.abspath(os.path.dirname(__file__))
42
43 # (tabla, columna) de todo lo que pasa a estar cifrado en reposo.
44 CAMPOS = [
45     ("usuarios", "nombre"),
46     ("usuarios", "correo"),
47     ("usuarios", "alias"),
48     ("movimientos", "nota"),
49     ("gastos", "descripcion"),
50 ]
51
52
53 def abrir(url: str):
54     """Devuelve (conexión, dialecto, marcador_de_parámetro)."""
55     if url.startswith("postgres"):
56         import psycopg2 # nota: F401 (solo si de verdad se usa Postgres)
57
58         return psycopg2.connect(url), "postgres", "%s"
59     import sqlite3
60
61     ruta = url.replace("sqlite://", "")
62     return sqlite3.connect(ruta), "sqlite", "?"
63
64

```

```

65 def columnas(cur, dialecto: str, tabla: str) -> set[str]:
66     if dialecto == "sqlite":
67         cur.execute(f"PRAGMA table_info({tabla})")
68         return {fila[1] for fila in cur.fetchall()}
69     cur.execute(
70         "SELECT column_name FROM information_schema.columns WHERE table_name = %s",
71         (tabla,),
72     )
73     return {fila[0] for fila in cur.fetchall()}
74
75
76 def main() -> int:
77     ap = argparse.ArgumentParser()
78     ap.add_argument(
79         "--url",
80         default=os.environ.get(
81             "DATABASE_URL", "sqlite:/// " + os.path.join(BASE_DIR, "questcash.db")
82         ),
83     )
84     ap.add_argument("--dry-run", action="store_true", help="no escribe nada")
85     args = ap.parse_args()
86
87     conn, dialecto, ph = abrir(args.url)
88     cur = conn.cursor()
89
90     print(f"Base de datos : {args.url}")
91     print(f"Dialecto      : {dialecto}")
92     print(f"Modo          : {'SIMULACIÓN (no escribe)' if args.dry_run else 'ESCRITURA'}")
93     print("-" * 66)
94
95     # ---- 1. Respaldo ----
96     if dialecto == "sqlite" and not args.dry_run:
97         ruta = args.url.replace("sqlite://", "")
98         copia = f"{ruta}.bak-{datetime.now():%Y%m%d-%H%M%S}"
99         shutil.copy2(ruta, copia)
100        print(f"Respaldo      : {copia}")
101
102    # ---- 2. Columna del índice ciego ----
103    if "correo_bi" not in columnas(cur, dialecto, "usuarios"):
104        print("Agregando columna usuarios.correo_bi ...")
105        if not args.dry_run:
106            cur.execute("ALTER TABLE usuarios ADD COLUMN correo_bi VARCHAR(64)")
107    else:
108        print("usuarios.correo_bi ya existe.")
109
110    # ---- 3. Ensachar columnas (solo Postgres) ----
111    if dialecto == "postgres" and not args.dry_run:
112        anchos = {"nota": 2048, "descripcion": 1024}
113        for tabla, col in CAMPOS:
114            cur.execute(
115                f"ALTER TABLE {tabla} ALTER COLUMN {col} TYPE VARCHAR({anchos.get(col, 512)})"
116            )
117        print("Columnas ensachadas a VARCHAR(512+).")
118
119    # ---- 4. Cifrado fila por fila ----
120    total_cifrados = 0
121    for tabla, col in CAMPOS:
122        cur.execute(f"SELECT id, {col} FROM {tabla}")
123        filas = cur.fetchall()
124        pendientes = [(i, v) for i, v in filas if v and not esta_cifrado(v)]
125        for id_, valor in pendientes:
126            if not args.dry_run:
127                cur.execute(
128                    f"UPDATE {tabla} SET {col} = {ph} WHERE id = {ph}", (cifrar(valor), id_)
129                )
130        total_cifrados += len(pendientes)
131    print(f" {tabla}. {col:<12} {len(pendientes):>4} de {len(filas):>4} filas cifradas")

```

```

132
133 # ---- 5. Índice ciego de los correos -----
134 # Se recalcula desde el valor en claro que se acaba de leer, por eso este
135 # paso va después del cifrado pero usa los datos originales en memoria.
136 cur.execute("SELECT id, correo FROM usuarios")
137 correos = cur.fetchall()
138 sin_indice = 0
139 for id_, valor in correos:
140     from crypto_utils import descifrar
141
142     claro = descifrar(valor)
143     bi = indice_ciego(claro)
144     if not args.dry_run:
145         cur.execute(f"UPDATE usuarios SET correo_bi = {ph} WHERE id = {ph}", (bi, id_))
146         sin_indice += 1
147     print(f" usuarios.correo_bi {sin_indice:>3} índices ciegos calculados")
148
149 if not args.dry_run:
150     try:
151         cur.execute(
152             "CREATE UNIQUE INDEX IF NOT EXISTS ux_usuarios_correo_bi ON usuarios (correo_bi)"
153         )
154         print("Índice UNIQUE ux_usuarios_correo_bi creado.")
155     except Exception as exc: # correos duplicados preexistentes
156         print(f" ! No se pudo crear el índice UNIQUE: {exc}")
157     conn.commit()
158
159 conn.close()
160 print("-" * 66)
161 print(f"Listo. {total_cifrados} valores cifrados.")
162 if args.dry_run:
163     print("(Simulación: no se escribió nada.)")
164 return 0
165
166
167 if __name__ == "__main__":
168     sys.exit(main())

```

Referencias

- [1] OWASP Foundation. *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [2] C. Percival, S. Josefsson. *The scrypt Password-Based Key Derivation Function*. RFC 7914, IETF, 2016.
- [3] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, 2007.
- [4] H. Krawczyk, P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, IETF, 2010.
- [5] Scott Arciszewski. *Building Searchable Encrypted Databases with PHP and SQL* — origen del patrón de índice ciego.